

M I C R O P R O C E S S O R A N D E M B E D D E D S Y S T E M S

SMART TRAFFIC LIGHT SYSTEM

NAUFAL SUL'AIM BIN MUHD KHAIRI
51221120232
L05-B01



PROJECT OBJECTIVES

Improved Traffic Flow

optimise traffic light timings by correctly recognising the presence of vehicles, decreasing congestion and delays at intersections.

Enhanced Safety

system ensures their safety by coordinating with traffic signals to provide a safe passage

Timer Display

display will show the remaining time for each traffic light phase, helping both cars and pedestrians to understand how much time is available for each light phase.

Microcontroller-Based Implementation

system's control unit will be the STM32 F401RE microcontroller. It provides the processing power, GPIO ports, and communication interfaces required to communicate with various components such as the ultrasonic sensors, pedestrian button, traffic lights, and 7-segment display.

OPERATIONAL OF THE APPLICATION

Vehicle Detection

At one junction, the smart traffic system detects oncoming vehicles using ultrasonic sensors.

Traffic Light Control

The traffic light control capability is in duty of managing the traffic light signals at each junction based on the detection of vehicles and pedestrian crossing requests.

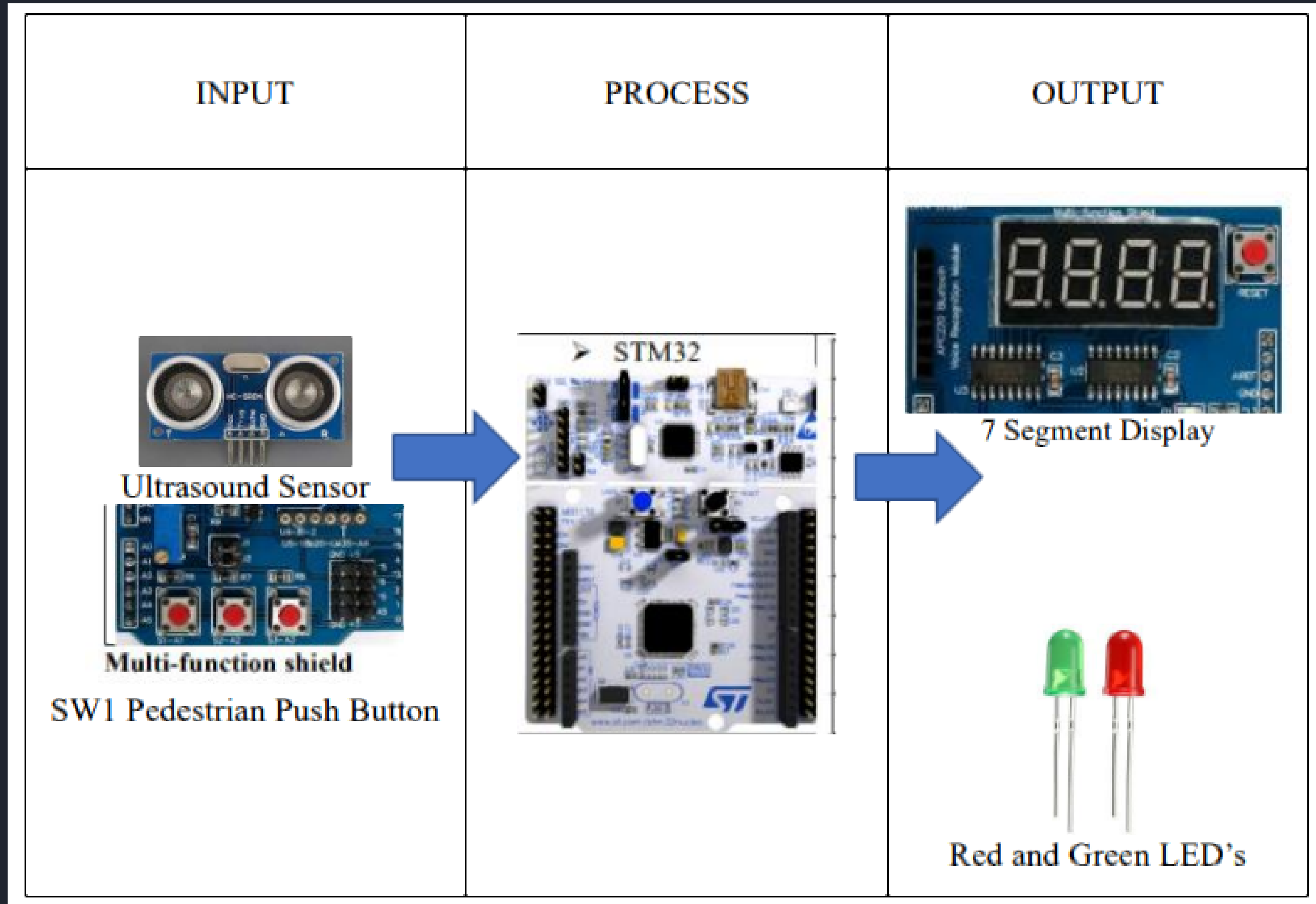
Pedestrian Crossing

pedestrian button as an interrupt to focus on pedestrian safety.

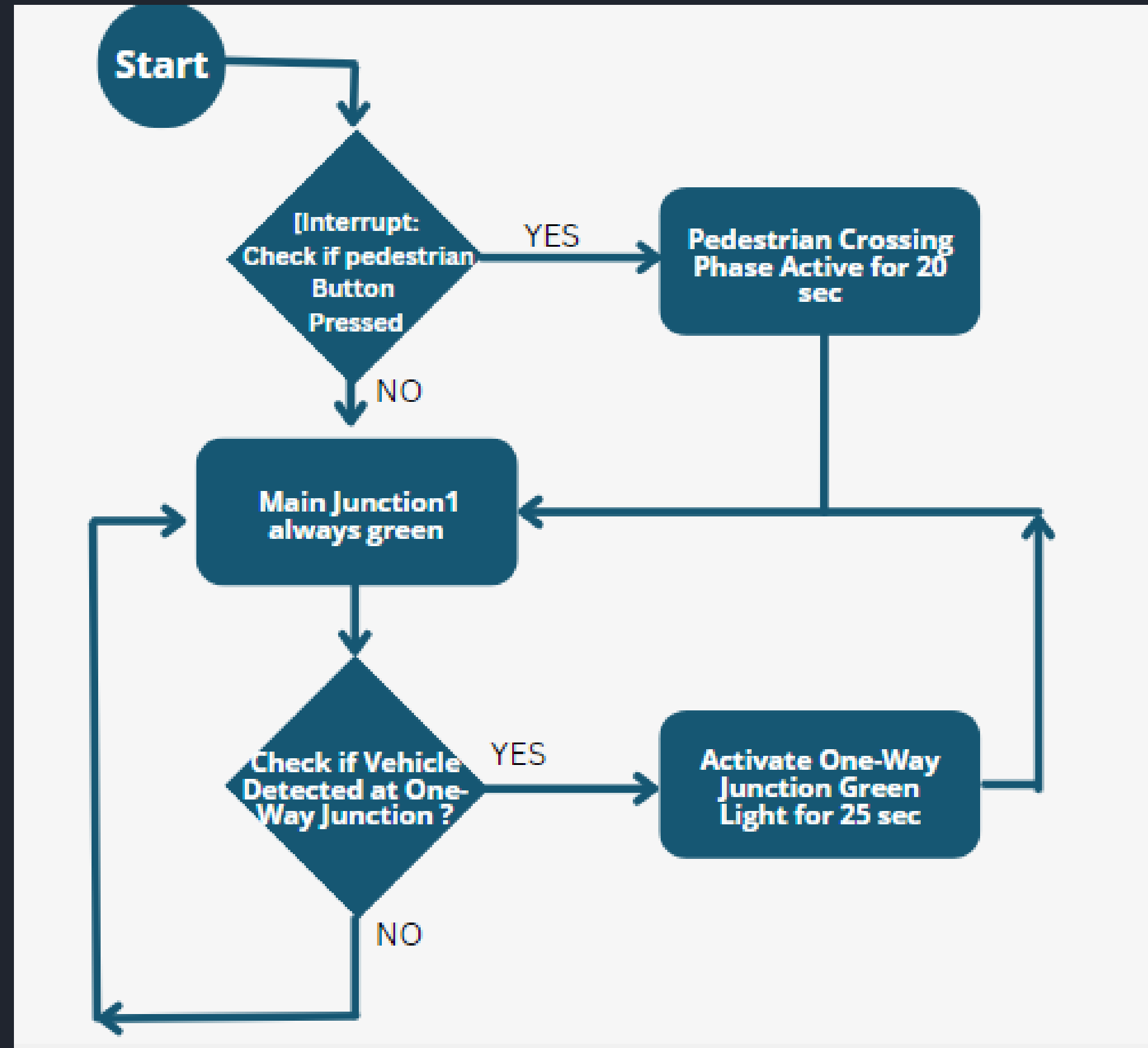
Counter Display

continuously updates the 7-segment display, representing the duration of each phase.

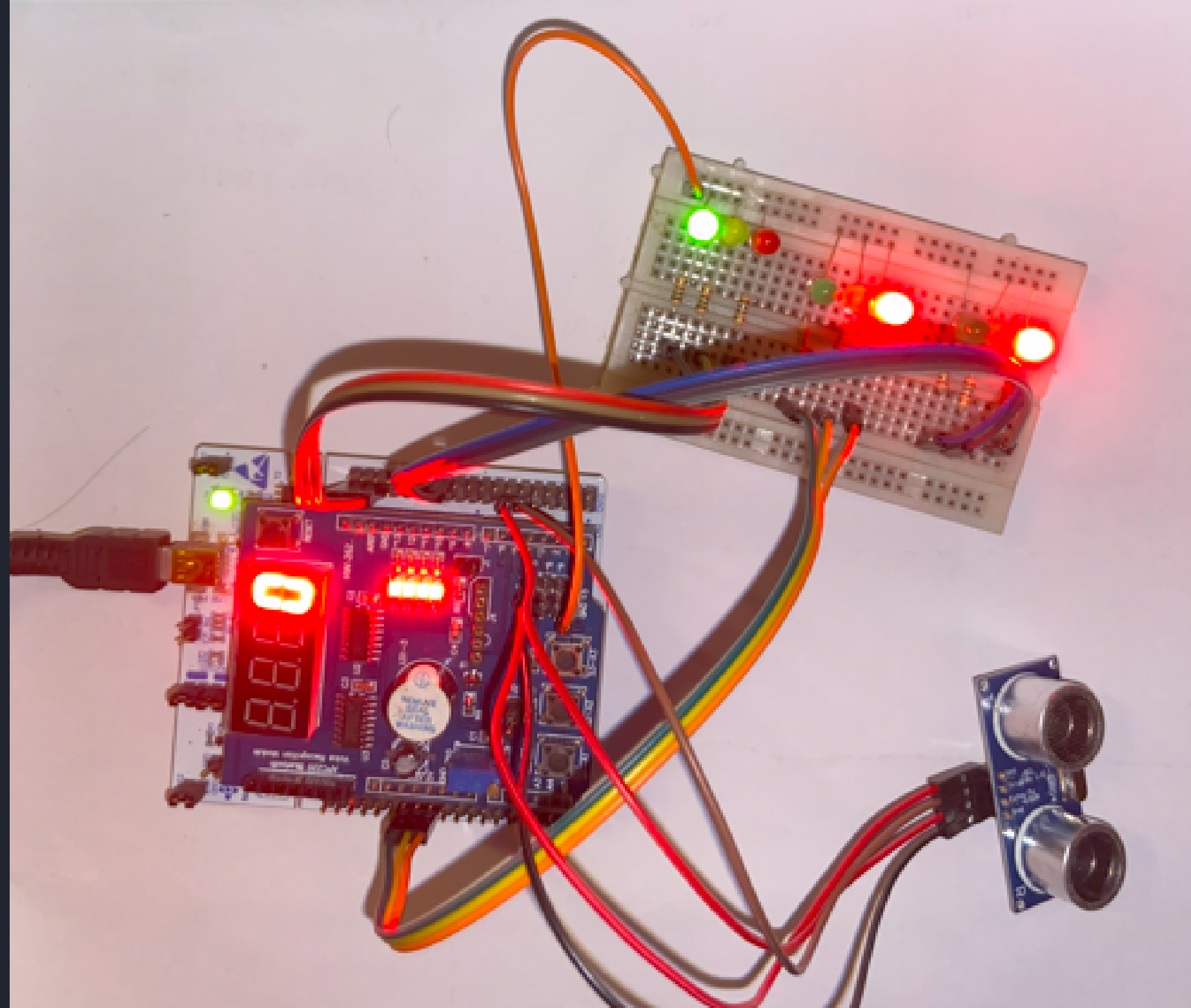
BLOCK DIAGRAM



Flowchart



Result



CODING

```
1  #include "mbed.h"
2  #include "hcsr04.h"
3  // pins for ultrasound sensor
4  HCSR04 sensor(PB_4, PB_10);
5
6  // pins for pedestrian button interrupt
7  InterruptIn pedestrianButton(PA_1); // Pedestrian button pin (GPIO interrupt)
8
9  //USB serial communication
10 Serial pc(USBTX, USBRX); // Define Serial object for USB communication
11
12 Timer carTimer; // Timer object to track the duration of the obstacle detection
13
14 // pins for traffic lights
15 DigitalOut junction1Green(PC_9); // Green LED for junction 1
16 DigitalOut junction1Yellow(PB_8); // Yellow LED for junction 1
17 DigitalOut junction1Red(PB_9); // Red LED for junction 1
18 DigitalOut junction2Green(PA_13); // Green LED for junction 2
19 DigitalOut junction2Yellow(PA_14); // Yellow LED for junction 2
20 DigitalOut junction2Red(PA_15); // Red LED for junction 2
21 DigitalOut PedestrianGreen1(PA_12); // GreenPedestrian LED for Pedetrian
22 DigitalOut PedestrianGreen2(PA_11); // GreenPedetrian LED for Pedestrian
23 DigitalOut PedestrianRed(PB_12); // RedPedestrian LED for Pedestrian
24
25 //Output for 7 segmment Counter Display
26 DigitalOut dataIn(PA_9);
27 DigitalOut serialClk(PA_8);
28 DigitalOut parallelClk(PB_5);
29 DigitalOut displayD1(PA_5);
30 DigitalOut displayD2(PA_6);
31 DigitalOut displayD3(PA_7);
32 DigitalOut displayD4(PB_6);
```

CODING

```
34 int num[11] = {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90, 0xFF};
35 int seg[4] = {0xF1, 0xF2, 0xF4, 0xF8};
36
37 int cnt = 0;
38 int16_t counter = 0;
39 int8_t d1, d2, d3, d4;
40
41 void LEDs(uint8_t x)
42 {
43     displayD1 = (x & (1 << 0)) ? 1 : 0;
44     displayD2 = (x & (1 << 1)) ? 1 : 0;
45     displayD3 = (x & (1 << 2)) ? 1 : 0;
46     displayD4 = (x & (1 << 3)) ? 1 : 0;
47 }
--
78 // Define variables
79 bool pedestrianCrossingActive = false; // Flag to track pedestrian crossing phase
80
81
82 void CarDetection()
83 {
84     long distance = sensor.distance(); // Get the distance from the ultrasound sensor
85     pc.printf("Distance: %ld cm\n", distance);
86
87     if (distance < 7) {
88         pc.printf("Car detected\n\r");
89         // Additional code for car detection action
90     } else {
91         pc.printf("No car detected\n\r");
92         // Additional code for no car detection action
93     }
94
95     wait(1);
96 }
```

```
49 void ledcnt(void)
50 {
51     LEDs(cnt);
52     wait_ms(500);
53     if (++cnt > 15)
54         cnt = 0;
55 }
56
57 void displaySeg(uint8_t data, uint8_t seg)
58 {
59     int8_t i;
60     uint16_t dataseg;
61     dataseg = (data << 8) + seg;
62
63     parallelClk = 0;
64     for (i = 15; i >= 0; i--)
65     {
66         serialClk = 0;
67         if (dataseg & (1 << i))
68             dataIn = 1;
69         else
70             dataIn = 0;
71         serialClk = 1;
72     }
73     parallelClk = 1;
74     wait_ms(1);
75 }
```


CODING

```
98 void TrafficJunction1Off()  
99  
100 {  
101  
102     pedestrianButton.disable_irq();  
103  
104     counter = 179;  
105  
106     while (counter > 170)  
107     {  
108         d1 = (counter / 10) % 10;  
109         d2 = (counter / 100) % 10;  
110         d3 = (counter / 1000) % 10;  
111         d4 = (counter / 10000) % 10;  
112         for (int x = 0; x < 20; x++)  
113         {  
114             displaySeg(num[d4], seg[0]);  
115             displaySeg(num[d3], seg[1]);  
116             displaySeg(num[d2], seg[2]);  
117             displaySeg(num[d1], seg[3]);  
118  
119             junction1Green = 0;  
120             junction1Yellow = 0;  
121             junction1Red = 0;  
122  
123             junction2Green = 0;  
124             junction2Yellow = 0;  
125             junction2Red = 1;  
126  
127             PedestrianGreen1 = 0;  
128             PedestrianGreen2 = 0;  
129             PedestrianRed = 1;  
130  
131         }  
132         counter--; // Decrement counter  
133     }
```

```
387     counter = 39;  
388  
389     while (counter > 10)  
390     {  
391         d1 = (counter / 10) % 10;  
392         d2 = (counter / 100) % 10;  
393         d3 = (counter / 1000) % 10;  
394         d4 = (counter / 10000) % 10;  
395         for (int x = 0; x < 20; x++)  
396         {  
397             displaySeg(num[d4], seg[0]);  
398             displaySeg(num[d3], seg[1]);  
399             displaySeg(num[d2], seg[2]);  
400             displaySeg(num[d1], seg[3]);  
401  
402             junction1Green = 0;  
403             junction1Yellow = 0;  
404             junction1Red = 1;  
405  
406             junction2Green = 0;  
407             junction2Yellow = 0;  
408             junction2Red = 1;  
409  
410             PedestrianGreen1 = 0;  
411             PedestrianGreen2 = 0;  
412             PedestrianRed = 1;  
413  
414         }  
415         counter--; // Decrement counter  
416     }  
417     pedestrianButton.enable_irq();  
418 }  
419
```

CODING

```
420 void TrafficJunction2On()  
421 // Junction 2: green for 25 sec  
422 {  
423     pedestrianCrossingActive = true;  
424  
425     counter = 380;  
426  
427     while (counter > 130)  
428     {  
429         d1 = (counter / 10) % 10;  
430         d2 = (counter / 100) % 10;  
431         d3 = (counter / 1000) % 10;  
432         d4 = (counter / 10000) % 10;  
433         for (int x = 0; x < 20; x++)  
434         {  
435             displaySeg(num[d4], seg[0]);  
436             displaySeg(num[d3], seg[1]);  
437             displaySeg(num[d2], seg[2]);  
438             displaySeg(num[d1], seg[3]);  
439  
440             junction1Green = 0;  
441             junction1Yellow = 0;  
442             junction1Red = 1;  
443  
444             junction2Green = 1;  
445             junction2Yellow = 0;  
446             junction2Red = 0;  
447  
448             PedestrianGreen1 = 0;  
449             PedestrianGreen2 = 0;  
450             PedestrianRed = 1;  
451         }  
452     }  
453     counter--; // Decrement counter  
454 }
```

```
458 void TrafficJunction2Off()  
459 {  
460     pedestrianButton.disable_irq();  
461  
462     counter = 129;  
463  
464     while (counter > 120)  
465     {  
466         d1 = (counter / 10) % 10;  
467         d2 = (counter / 100) % 10;  
468         d3 = (counter / 1000) % 10;  
469         d4 = (counter / 10000) % 10;  
470         for (int x = 0; x < 20; x++)  
471         {  
472             displaySeg(num[d4], seg[0]);  
473             displaySeg(num[d3], seg[1]);  
474             displaySeg(num[d2], seg[2]);  
475             displaySeg(num[d1], seg[3]);  
476  
477             junction1Green = 0;  
478             junction1Yellow = 0;  
479             junction1Red = 1;  
480  
481             junction2Green = 0;  
482             junction2Yellow = 0;  
483             junction2Red = 0;  
484  
485             PedestrianGreen1 = 0;  
486             PedestrianGreen2 = 0;  
487             PedestrianRed = 1;  
488         }  
489     }  
490     counter--; // Decrement counter  
491 }
```

```
621 counter = 39;  
622  
623 while (counter > 10)  
624 {  
625     d1 = (counter / 10) % 10;  
626     d2 = (counter / 100) % 10;  
627     d3 = (counter / 1000) % 10;  
628     d4 = (counter / 10000) % 10;  
629     for (int x = 0; x < 20; x++)  
630     {  
631         displaySeg(num[d4], seg[0]);  
632         displaySeg(num[d3], seg[1]);  
633         displaySeg(num[d2], seg[2]);  
634         displaySeg(num[d1], seg[3]);  
635  
636         junction1Green = 0;  
637         junction1Yellow = 0;  
638         junction1Red = 1;  
639  
640         junction2Green = 0;  
641         junction2Yellow = 0;  
642         junction2Red = 1;  
643  
644         PedestrianGreen1 = 0;  
645         PedestrianGreen2 = 0;  
646         PedestrianRed = 1;  
647     }  
648     counter--; // Decrement counter  
649 }
```

CODING

```
657 void PedestrianOn()
658
659 {
660     // Pedestrian: Green for 15 seconds
661
662     counter = 209;
663
664     while (counter > 200)
665     {
666         d1 = (counter / 10) % 10;
667         d2 = (counter / 100) % 10;
668         d3 = (counter / 1000) % 10;
669         d4 = (counter / 10000) % 10;
670         for (int x = 0; x < 20; x++)
671         {
672             displaySeg(num[d4], seg[0]);
673             displaySeg(num[d3], seg[1]);
674             displaySeg(num[d2], seg[2]);
675             displaySeg(num[d1], seg[3]);
676
677             junction1Green = 0;
678             junction1Yellow = 0;
679             junction1Red = 1;
680
681             junction2Green = 0;
682             junction2Yellow = 0;
683             junction2Red = 1;
684
685             PedestrianGreen1 = 1;
686             PedestrianGreen2 = 0;
687             PedestrianRed = 0;
688
689         }
690         counter--; // Decrement counter
691     }
692 }
```

```
counter = 59;

while (counter > 10)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}
```

```
1161
1162 void Pedestrianplay()
1163 {
1164     if (pedestrianCrossingActive) {
1165         // Pedestrian button pressed during pe
1166         TrafficJunction20ff();
1167         PedestrianOn();
1168     } else {
1169         // Pedestrian button pressed during no
1170         TrafficJunction10ff();
1171         PedestrianOn();
1172     }
1173 }
1174 void junction2()
1175 {
1176     TrafficJunction10ff();
1177     TrafficJunction20n();
1178     TrafficJunction20ff();
1179 }
1180
```


CODING

```
1181 void normal(float timer)
1182 {
1183     pedestrianCrossingActive = false;
1184
1185     junction1Green = 1;
1186     junction1Yellow = 0;
1187     junction1Red = 0;
1188
1189     junction2Green = 0;
1190     junction2Yellow = 0;
1191     junction2Red = 1;
1192
1193     PedestrianGreen1 = 0;
1194     PedestrianGreen2 = 0;
1195     PedestrianRed = 1;
1196
1197     displaySeg(num[0], seg[0]);
1198     displaySeg(num[0], seg[1]);
1199     displaySeg(num[0], seg[2]);
1200     displaySeg(num[0], seg[3]);
1201
1202 }
```

```
1203 void obstacleDetection()
1204 {
1205     float obstacleTimer = 0.0; // Variable to track the duration of obstacle detection
1206
1207     while (1)
1208     {
1209         // Check if obstacle is detected
1210         if (sensor.distance() < 7)
1211         {
1212             if (obstacleTimer == 0.0)
1213             {
1214                 carTimer.reset(); // Reset the timer when the obstacle is first detected
1215                 carTimer.start(); // Start the timer
1216             }
1217
1218             // Update the obstacleTimer with the elapsed time
1219             obstacleTimer = carTimer.read();
1220
1221             // Check if the obstacle is detected for more than 5 seconds
1222             if (obstacleTimer >= 5.0)
1223             {
1224                 junction2(); // Call the junction2 function
1225                 break; // Exit the obstacle detection loop
1226             }
1227         }
1228         else
1229         {
1230             obstacleTimer = 0.0; // Reset the obstacleTimer if no obstacle is detected
1231         }
1232
1233         normal(1000.0); // Call the normal function
1234
1235         wait(1.0); // Delay for 1 second
1236     }
1237 }
1238
```

CODING

```
1239  int main()
1240  {
1241      pedestrianButton.fall(&Pedestrianplay);
1242
1243
1244      while (1)
1245      {
1246          {
1247              normal(1000);
1248          }
1249
1250          CarDetection();
1251          obstacleDetection(); // Call the obstacleDetection function
1252      }
1253  }
```